

LABORATORY OBSERVATION

Student Name: K. Hansikha

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5-2

1. TITLE

Node.js File Management

2. AIM

To develop a terminal-based Node.js application that accepts a filename and text, stores the initial data in a file, reads the stored data, appends another message, and displays the completed file contents.

3. OBJECTIVE

The objectives of this experiment are:

- To use Node.js built-in modules for terminal input and file manipulation.
 - To collect filename and content values from the user through readline.
 - To create and write a text file asynchronously.
 - To retrieve existing file data and display it in the terminal.
 - To append new information while preserving the original text.
 - To handle file errors and close the readline interface correctly.
-

4. THEORY

The File System module (fs) is a core Node.js module that provides methods for working with files. The program in this experiment uses asynchronous callbacks so that the next step runs after the current file operation completes.

The readline module creates an interactive terminal interface. The user first supplies the file name and initial content. `writeFile()` stores the first message, `readFile()` retrieves it, `appendFile()` adds a second message, and the last `readFile()` prints the final contents.

Each callback receives an error value. The program checks this value before continuing and closes readline when an error occurs or when all operations have completed.

5. ALGORITHM / PROCEDURE

1. Import fs and readline.
2. Create the readline interface for terminal input and output.
3. Request the filename from the user.
4. Request the first content string.
5. Write the first content to the selected file with `writeFile()`.
6. Check for a writing error and stop safely when necessary.
7. Read the file with `readFile()` and show the initial data.
8. Ask for additional content.
9. Append the new content on a new line with `appendFile()`.
10. Read the file again and display the final text, then close readline.

6. PROGRAM

fileManager.js

```
const fs = require("fs");
const readline = require("readline");

// Create readline interface
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Ask for filename
rl.question("Enter filename: ", (filename) => {

  // Ask for initial content
  rl.question("Enter content: ", (content) => {

    // Create/write file
    fs.writeFile(filename, content, "utf8", (err) => {

      if (err) {
        console.log("Error creating file:", err);
        rl.close();
        return;
      }

      console.log("File created successfully.");

      // Read file
      fs.readFile(filename, "utf8", (err, data) => {

        if (err) {
          console.log("Error reading file:", err);
          rl.close();
          return;
        }

        console.log("\nInitial file contents:");
        console.log(data);

        // Ask for additional content
        rl.question("\nEnter additional content: ", (additionalContent) => {

          // Append content
          fs.appendFile(
            filename,
            "\n" + additionalContent,
            "utf8",
            (err) => {

              if (err) {
                console.log("Error appending file:", err);
                rl.close();
                return;
              }

              console.log("Content appended successfully.");

              // Read final contents
              fs.readFile(filename, "utf8", (err, finalData) => {

                if (err) {
                  console.log("Error reading final file:", err);
                  rl.close();
                  return;
                }

                console.log("\nFinal file contents:");
                console.log(finalData);
                rl.close();
              });
            });
        });
      });
    });
  });
});
```

7. CODE EXPLANATION

Code / Method	Explanation
require("fs")	Loads the built-in File System module used for writing, reading, and appending files.
require("readline")	Loads the module that accepts interactive input from the terminal.
readline.createInterface()	Connects the program to standard input and output streams.
rl.question()	Prompts the user and passes the entered value to a callback function.
fs.writeFile()	Creates or overwrites the named file with the initial content.
fs.readFile()	Reads the current file data using UTF-8 text encoding.
fs.appendFile()	Adds the additional text to the end of the existing file.
if (err)	Checks whether the corresponding file-system operation failed.
rl.close()	Terminates the readline interface after the task is finished.

8. TEST CASES AND EXECUTION DATA

Test Case	Input	Expected Result	Actual Result	Status
TC-01	student.txt; initial text; additional text	File is created, read, updated, and final text is shown.	All four stages completed successfully.	Pass
TC-02	Empty filename	A suitable file-input error should be shown.	Not executed in the demonstrated run.	-
TC-03	Additional content entered after initial read	The second message should appear after the first message.	Second message appended successfully.	Pass
TC-04	File operation failure	The error should be displayed and execution should close safely.	Not executed in the demonstrated run.	-

9. OUTPUT

Output 1: Terminal execution of the asynchronous Node.js file-management program

```
PS C:\Users\hansi\OneDrive\Desktop\git-programs\full-stack\PBL\Q2_Node_File_Management> node fileManager.js
Enter filename: student.txt
Enter content: My name is Hansikha
File created successfully.

Initial file contents:
My name is Hansikha

Enter additional content: I am student at ANITS
Content appended successfully.

Final file contents:
My name is Hansikha
I am student at ANITS
```

The supplied execution output shows the entered filename student.txt, the initial message "My name is Hansikha", successful file creation, reading of the first contents, appending of "I am student at ANITS", and the final two-line file contents.

The demonstrated run matches the source program: `writeFile()` creates the file, `readFile()` displays the first version, `appendFile()` adds the second line, and the final `readFile()` displays the completed content.

10. RESULT

Thus, the Node.js File Management application was implemented successfully using the fs and readline modules. The demonstrated output confirms that the program accepts terminal input, creates the requested file, reads its contents, appends new information, and displays the final file data without changing the original message.

The experiment also demonstrates asynchronous callback handling and error checking for each file operation. The readline interface is closed after the final output, completing the execution cleanly.

How this would be submitted.

Source Code

Q2_Node_File_Management/

- fileManager.js

- student.txt